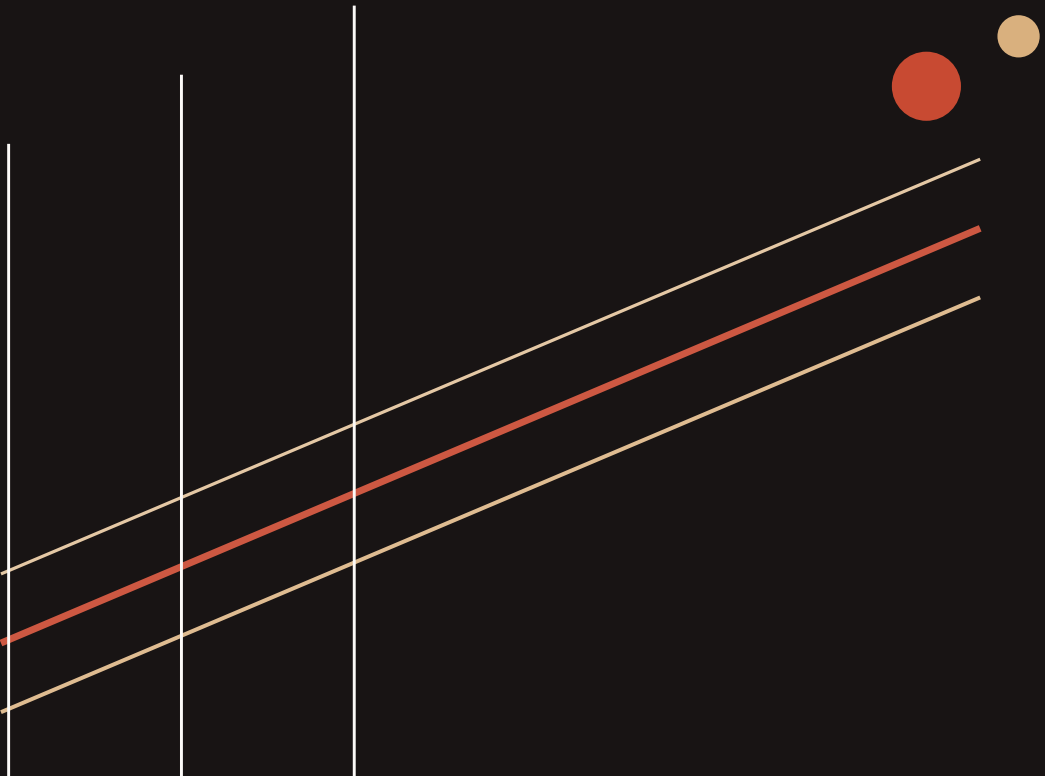


VIBE CODING IN THE AI ERA

Vibe Coding Tauri 2

リクエスト・ビルド・検証—AIとともに6つのアプリを完成させる

Request | Build | Verify | Repeat



Ted Park

2026

Vibe Coding Tauri 2

リクエスト・ビルド・検証—AIとともに6つのアプリを完成させる

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Hiragino Kaku Gothic ProN | Menlo.

目次

1 序文	1
1.1 なぜこの書籍を書いたのか	1
1.2 この書籍が扱うもの・扱わないもの	2
1.3 書籍のルーブリークエスト、構築、検証	3
1.4 4つのアプリと2つのTUIプロジェクトの概要	3
1.5 この書籍の読み方	4
1.6 全体の道のり	5
1.7 前提知識	6
1.8 開発環境	6
1.9 著者より	7
2 Chapter 1. なぜAIで構築するのか——一人では構築できないアプリ	8
2.1 学習目標	8
2.2 1.1 私は忙しかったが、それでも4つを出荷した	9
2.3 1.2 なぜ今なのか——2026年が変曲点である理由	10
2.4 1.3 以前の開発：私がコードを書いていた	10
2.5 1.4 現在の開発：AIがコードを書き、私が決定	11
2.6 1.5 「AI コードを読むだけで十分」の現実	12
2.7 1.6 役割の転換：コードレビュー者から要件改良者へ	13
2.8 1.7 シーン——readbooks.aiからのPDFパーサーエピソード	14
2.9 1.8 一緒に構築する6つのプロジェクト	15
2.9.1 1. Mandai —生産性ツリー + AIコーチ	15
2.9.2 2. readbooks.ai —多言語PDF翻訳	15
2.9.3 3. トレーディングダッシュボード—19画面リアルタイムダッシュボード	16

2.9.4	4. RL & バックテストラボ—長時間実行Pythonプロセスコントローラー	16
2.9.5	5. TUIトレーディングモニター—ターミナルダッシュボード	16
2.9.6	6. TUIログビューアー—アイデアからツールまで2時間	17
2.10	1.9 読者が持ち帰るもの	17
2.11	1.10 2年間、数値で	18
2.12	1.11 始める前のセルフチェック	18
2.13	1.12 この本がカバーしないこと	19
2.14	要約	19
2.15	1.12 この書籍がカバーしないもの	20
2.16	まとめ	21

CHAPTER 1

序文

1.1 なぜこの書籍を書いたのか

2026年、私は一人でTauri 2アプリを4つ構築しました。生産性アプリが1つ、サービスとして販売しているWebアプリが1つ、ライブトレーディングシステムに接続されたダッシュボードが1つ、そして強化学習・バックテストラボが1つです。すべて一人で、最初から最後まで作り上げました。

注目すべき点は、私は時間に余裕のある開発者ではないということです。本業があります。家には子供がいます。平日の夕方に1〜2時間、週末に数時間程度しか時間が取れません。以前の時代なら、このような時間的制約では一つのアプリを作るのがやっとでした。しかし2026年に私は4つのアプリを出荷しました。一つのことが変わったからです—**AIエージェント**です。Claude Codeにコンテキストを与えると、夕食後の30分間でデータベーススキーマが出来上がります。翌朝、出勤前の15分でバグが修正されます。エージェントがタイプします。私はリクエストと検証のみを行います。

もしこの文章を2020年の私に見せたら、嘘だと言われたでしょう。2020年の私は一つのアプリに6か月をかけていました。設計に2週間、実装に3か月、バグ修正に1か

月、デプロイに1か月。年間2つのアプリが限界でした。本業と子育てをしながら副業でアプリを作るなど考えられませんでした。

変わったのは私ではありません。技術スタックはより困難になりました。Rustを書き、WebSocketsを接続し、DuckDBを組み込み、TauriでPythonプロセスを起動させています。以前なら、これらの要素それぞれに数週間かかっていたでしょう。

唯一変わったことは、**私が自分でコードをタイプしなくなった**ということです。

AI時代において、「手でコードを書く」ことは目標ではありません—それは多くの手段の一つに過ぎません。手段は委任できます。私がリクエストを出し、結果を読み、間違っている部分を指摘し、再度依頼します。この循環を一日の隙間時間に実行します。数か月後、4つのアプリが存在しています。これが機能したのは**時間があったからではなく、時間がなかったために**エージェントが私を支援してくれたからです。

この書籍はそのループの記録です。コードのウォークスルーではありません。**AIとの働き方**を記録したものです。

1.2 この書籍が扱うもの・扱わないもの

以下がこの書籍で扱う内容です。

- ・ AIに対するリクエストの枠組み方。コンテキスト、制約、目標、形式の分離方法。
- ・ 返ってきた結果の検証方法。「動作する」と「正しい」の違い。
- ・ 同じリクエストの再実行方法。1回で十分でない場合のループの効率化。
- ・ 6つのプロジェクトにおいて、AIが正しかった場面と間違った場面。私が手動で上書きしなければならなかった箇所。
- ・ 実際のプロンプトテンプレート、スラッシュコマンド、CLAUDE.md ファイル。

以下は扱わない内容です。

- ・ Rust構文チュートリアル。Result<T, E>については説明しません。
- ・ Svelte 5構文チュートリアル。\$stateと\$effectは使用しますが、ゼロから教えることはしません。
- ・ 公式Tauri 2ドキュメントのコピー。物語に必要な最小限の内容のみに触れます。

- ・一般的なプロンプトエンジニアリング。執筆、画像生成、RAGはこの書籍の範囲外です。
- ・AIモデルの内部構造。ClaudeとGPTのトークナイザーの比較などはいけません。

この境界線をこれほど明確にするのには理由があります。読者がこの書籍を読み終えて**自分の手で**自分のアプリを構築するためには、寄り道をしている余裕がないからです。

1.3 書籍のループ—リクエスト、構築、検証

この書籍が継続的に実行するループには3つの段階があります。**リクエスト**は私の意図をAIが理解できる言語に翻訳することです。**構築**はAIがコードを出力し、私がそれを貼り付けて実行することです。**検証**は動作確認ではありません—正しいかどうかを確認することです。検証で見つかった問題はリクエストに戻されて、1周が完了します。私は1つの機能につき平均3〜5周実行します。正確な定義と段階別チェックリストは第2章にあります。

1.4 4つのアプリと2つのTUIプロジェクトの概要

6つのプロジェクトは意図的に選ばれています。それぞれがAIコーディングの異なる側面を表面化します。

Mandai —Mandal-Art (9×9目標グリッド) + GTD + Pomodoro + AIコーチ。データはローカルDuckDBに保存され、OpenAI、Claude、Gemini、Groqが単一のコマンドの背後に位置しています。テーマは**マルチプロバイダーストリーミングとストリームキャンセル**です。AIが最も見逃しがちだった箇所は、前のストリームをキャンセルする部分でした。

readbooks.ai—PDFを36言語に翻訳するライブサービス。Next.jsからSvelteKit 5に移植しました。この章では、**フレームワーク移植**をどこまでAIに任せられるかテストしました。結論：100%ではありません。

トレーディングダッシュボード—19画面のリアルタイムダッシュボード。Python FastAPI + WebSocket、AtomicU16プロキシポート、Cloudflare Tunnel、ローカルチャート。テーマは**大規模アプリを自分とAIの間でどう分割するか**です。

RL&バックテストラボ—TauriからPython強化学習トレーニングとバックテストを起動し、stdoutをストリーミングします。**長時間実行される外部プロセス制御**です。この領域でAIは最も弱い性能を示しました。プロンプトにkill -9を明記してから、ようやくコードがそれを処理するようになりました。

TUIトレーディングモニター—ratatuiで構築されたターミナルベースのダッシュボード。リアルタイムポジション、P&L、アラートをターミナルでレンダリング。テーマは**TUIの出荷速度**と、なぜそれが自分用ツールの正当な代替手段となるのかです。

TUIログビューアー—強化学習トレーニングとバックテスト出力用のターミナルベースログビューアー。フィルター、検索、ライブテーリング機能。テーマは**アイデアから動作するツールまでの最速パス**としてのTUIです。ブラウザも、WebViewも、バンドリングも不要—cargo runで動作します。

6つのプロジェクトを順番に読む必要はありません。気になるものを選んでください—ループはどこでも同じ形をしています。

1.5 この書籍の読み方

パート I（第1～5章）は順序通りに読むことをお勧めします。前提、ループの定義、VSCode Copilotのワークフロー、ツールのセットアップ、そして書籍全体のTauri 2基礎が含まれています。これら5章をスキップすると、後のプロンプト設計の議論が宙に浮いてしまいます。

パート II～VI（第6～17章）はアプリ別に読むことができます。各パートは独立しています。Mandaiに興味があるなら、パート IIのみ読んでください。トレーディングダッシュボードに興味があるなら、パート IVのみ読んでください。ターミナルアプリが意

外に実用的であることを見たいなら、パート VI にジャンプしてください。各パートは「このアプリから学ぶループパターン」から始まります。

最終パート VII（第18章）は、この手法を自分のアプリに適用する際に戻ってくる章です。最初の通読時に一度読み、実際に自分のアプリを作り始めるときに再度読んでください。テンプレートと失敗パターンが含まれています。

1.6 全体の道のり

パート	章	テーマ
I	1 –5	なぜAIなのか、ループ、Copilotワークフロー、ツール、Tauri 2 基礎
II	6 –9	Mandai —設計、DB、AI API、落とし穴
III	10 –11	readbooks.ai —プロンプティング、検証
IV	12 –14	トレーディングダッシュボード —スケール、リアルタイム、デプロイ
V	15	RL & バックテスト ラボードメイン知識の注入
VI	16 –17	TUIアプリ —高速出荷するターミナルUI
VII	18	自分のアプリへのこの手法の適用

1.7 前提知識

この書籍を読み進めるために、大体以下の背景知識が必要です。

- ・ **RustとTypeScriptを読む能力。** 書く必要はありません。AIが生成したコードを見て?演算子が何をするか説明できれば十分です。
- ・ **Gitとターミナルでの作業に慣れていること。** ブランチ作成とコミット分割を習慣的に行っているなら、この書籍のループをそのままワークフローに組み込みます。
- ・ **AIコーディングツールの実践経験。** Claude Code、Cursor、GitHub Copilot、ChatGPTの少なくとも一つを実際のプロジェクトで使ったことがあるべきです。使ったことがなければ、まず1週間トイプロジェクトでAIを試してから、この書籍に戻ってください。

逆に、以下は**必要ない**ものです。Tauri 1の経験、SvelteKitの経験、本格的なRustの経験はすべて不要です。必要な内容は各章内で説明します。

1.8 開発環境

この書籍のすべてのコードとスクリーンショットは、以下の環境で作成されました。

項目	バージョン
OS	macOS 15.1 (Sequoia)
Rust	1.87.0
Node.js	22.11 LTS
pnpm	9.15
Tauri	2.1
SvelteKit	2.x + Svelte 5.1
DuckDB	1.1
Python	3.12 (RL Labのみ)

項目	バージョン
Claude Code	2026年Q1時点の最新版
Cursor	0.45

ほとんどがWindowsとLinuxでも同様に動作します。動作しない場合は、該当章内で違いを示します。

1.9 著者より

この書籍のすべての内容は、私が実際に経験したものです。うまくいかなかったループも、うまくいったループと並べて残しました。AIと3回格闘してから自分でコードを修正したシーン、AIがもっともらしく間違ったコードを生成したのに気づかずに通してしまったシーン、同じミスを2回犯したシーンがあります。何も綺麗に磨き上げられていません。この荒々しさがこの書籍の素材です。

本業を持ち、子育てをしながら、それでもソロ開発者として4つのTauri 2アプリを出荷した人間の作業記録として読んでください。時間があつたから書かれた書籍ではありません。時間がなくても—AIエージェントが可能にしてくれた記録です。そして5つ目のアプリは、あなたが構築するものです。

テッド・パーク

2026年春、ソウルにて

Chapter 1. なぜAIで構築するのか——一人では構築できないアプリ

2.1 学習目標

- ・ 2026年に、本業と家での子育てを抱えた開発者がなぜ4つのデスクトップアプリを出荷できるのかを理解します。
- ・ 「すべてをタイプする」時代と「AIに委任する」時代の生産性の差を、数字と体験の両面で感じます。
- ・ 「AI生成コードを読めればよい」という主張がどこで真実で、どこで嘘になるかを知ります。

- ・ 開発者の役割が「コード作成者」から「要件改良者プラス検証者」への転換を受けます。
 - ・ 私と一緒に構築する6つのプロジェクトと、最終的に手に入る能力をプレビューします。
-

2.2 1.1 私は忙しかったが、それでも4つを出荷した

2026年に私は4つのTauri 2デスクトップアプリを出荷しました。どれも玩具ではありません。1つは毎日使用している生産性アプリ。1つはサービスとして販売されているもの。残り2つはライブトレーディングシステムに接続されています。全て一人で。

重要な前提：私は時間に余裕のある開発者ではありません。本業があります。家には子供がいます。平日の夜に1〜2時間、週末に数時間—それが全予算です。フリーランサーではありません。フルタイムのインディ開発者でもありません。普通のサラリーマンであり、仕事の後には子育てが待っています。

これらの条件下で、2020年の私なら4つのアプリを出荷することは不可能でした。

2020年、Tauri 1はまだアルファ段階で、SvelteKitにはまだ名前がなく、Rustは私が使用していない言語でした。何よりも、2020年の私は一度に6ヶ月間単一のプロジェクトにしがみついていた。ビジュアルに1週間、機能設計に2週間、実装に3ヶ月、バグ修正に1ヶ月、デプロイに1ヶ月。そのペースでは年間2つのアプリが上限でした—それも業務外の時間を注ぎ込むことができたときのみ。

2026年、同じ人間が**より厳しい条件下で**4つを出荷しました。スタックはより困難になりました。バックエンドにRustを書き、WebSocketsを処理し、DuckDBを埋め込み、Pythonプロセスを起動します。より困難な作業、より少ない時間、より多い出力。

変わったのは一つだけです。**私はもうほとんどタイプしません。**AIエージェントがタイプします。夕食後の30分の隙間にClaude Codeにコンテキストを投げ込むと、翌朝出勤前の15分で結果を検証します。

2.3 1.2 なぜ今なのか—2026年が変曲点である理由

なぜ**この瞬間**に忙しいサラリーマン開発者が4つのアプリを出荷できるようになったのか？技術的背景を簡単に紹介します。

- ・ **会話型コーディングツールの成熟。** 2024年後期から、ClaudeやGPT-4クラスのモデルが複数ファイルのコンテキストを確実に処理し始めました。2026年までにはエージェントとツール使用がその上に着地し、「ファイルを読み、ファイルを書き、テストを実行」がすべて1つのセッションで起こります。
- ・ **Tauri 2 GA。** Tauri 2は2026年に安定版になりました。Rustバックエンド+Webフロントエンドの組み合わせがついに本格運用可能になりました。
- ・ **SvelteKit 5のrunes。** リアクティブ状態管理がクリーンな言語レベルの答えを得ました。AI生成フロントエンドコードの品質のばらつきが劇的に減少しました。

この3つすべてが2026年に収束しました。いずれか1つでも1~2年遅れていたら、私は4つのアプリを出荷していなかったでしょう。

2.4 1.3 以前の開発：私がコードを書いていた

10年前の私の一日をまだ再構築できます。朝にJIRAチケットを読みます。昼食後にエディターを開きます。Stack Overflowのタブを7つ開いていました。useStateを手でタイプしました。誤字を修正しました。Reactのライフサイクルを暗記していました。午後7時までに単一のコンポーネントが完成していました。

当時の私の最高のツールは**オートコンプリート**でした。最初の半分をタイプすると、IntelliSenseが`Array.prototype.filter`の後半を補完してくれました。それが限界でした。

コードの90%が私の指を通過しました。関数名、変数名、型宣言、繰り返しのif/else分岐、import文。すべて私がタイプしました。何がうまくいかないのでしょうか？すべてがうまくいかない可能性があります。コンパイラーは誤字をキャッチしましたが、誰も悪い設計をキャッチしませんでした。

そこで私は**設計をおろそかにし、タイピングで補いました**。考えるために一時停止する時間が長いほど、出荷が遅れました。「まず動かし、後でリファクタリング」がデフォルトでした。後は決して来ませんでした。

2.5 1.4 現在の開発：AIがコードを書き、私が決定

2026年は異なります。

朝にアイデアをメモに書き留めます。「Mandal-Art目標ツリー + Pomodoro + AIコーディング -> ローカルDuckDBに保存 -> すべてをTauri 2でラップ」午後にClaude Codeターミナルを開きます。プロンプトをタイプします。

Tauri 2 + SvelteKit 5を使用してMandal-Art 9x9グリッドコンポーネントを構築してくかセルをクリックすると編集モーダルが開きます。中央のセルが変更されたとき、AIに周囲の空のセルの候補を提案させます。

Svelte 5 runesのみ。ファイルごとにコードを分割してください。

Claudeは3分で、コンポーネント、ストア、AI提案用のTauriコマンドのドラフトを返します。私はそれを読みます。2つのことが気になります。Svelte 5構文が実際にはSvelte 4であることと、AI呼び出しがフロントエンドから実行されることです。プロンプトを追加します。

2つの修正。

1. `$state`、`$derived`、`$effect`のみを使用。``let``で宣言されたリアクティブ変数は使わ
 2. AI呼び出しをTauriコマンドに移動。APIキーはフロントエンドに公開されてはいけな
- さらに3分。パス。コミット。

以前なら6時間かかったタスクが20分で完了しました。私の手が行ったことは3つにまとめられます。**言葉で何が欲しいかを表現した。結果を読んだ。何が間違っているかを指摘した。**

同じMandaiプロジェクトについて、**AIが最初に成功したもの、間違っしたもの、修正させたもの**の表です。その感覚を持たない読者への参考として。

カテゴリー	例
正しかった	9×9グリッドレイアウト、Tailwindクラス構成、キーボードナビゲーションの骨格
間違った	Svelte 4ストア構文を混在、フロントエンドから直接AI呼び出し、anyを過度使用
修正させた	\$state/\$derivedで統一、AI呼び出しをTauri コマンドに移動、型をMandaiErrorに統合

この3行の表がすべての機能で繰り返されます。私の一日をこの表を埋めることとして公正に説明できるでしょう。

2.6 1.5 「AI コードを読むだけで十分」の現実

その主張は半分正しいです。

真の半分：タイピングドリルは時代遅れです。構文の暗記はそれほど重要ではありません。RustのResult<T, E>のすべての分岐を頭に入れておく必要はありません。AIが書き、あなたが読みます。ある箇所で立ち止まって「ここで?が正しい選択か?」と尋ねることができれば、それで十分です。

偽の半分：「読める」ための基準は今やはるかに高くなっています。

AIがこれを渡したとします。

```
#[tauri::command]
async fn get_candles(symbol: String) -> Result<Vec<Candle>,
String> {
    let conn = DB.lock().unwrap();
    let rows = conn.prepare("SELECT * FROM candles WHERE symbol
= ?")?;
    Ok(rows.query_map([symbol], |r|
Candle::from_row(r)).collect())
}
```

あなたは3秒以内に**これがなぜ間違っているかを見抜く**必要があります。unwrap()はパニックを起こす可能性があります。?はrusqlite::ErrorをResult<_, String>に自動変換しません。async関数で同期ミューテックスを保持するとランタイムが停止します。

AI生成コードを**読むだけ**の読者はこのバグを見落としします。実行し、何もクラッシュしないので、先に進みます。3週間後、本格運用データベース接続がブロックされ、アプリ全体がフリーズします。

読み取り基準の向上は一つのことを意味します。**コードを手で書きながら学んでいたトラップを今は意図的に学ばなければなりません**。これは私の後輩が最もよく滑り落ちる場所です。

2.7 1.6 役割の転換：コードレビュー者から要件改良者へ

2020年の日中に最も多く行っていたことは**コードを書く**ことでした。2026年の日中に最も多く行うことは**プロンプトを書く**ことです。

プロンプトを書くことは要件を書くことです。PMがConfluenceに書き留めるような要件ではありません。AIへのプロンプトははるかに**正確**でなければなりません。人間は「適当に判断して」を受け入れます。AIはそうしません。より正確には：そうしているように見えて、3時間後に関係のない何かを生産しています。

そこで毎日これらを書きます。

- ・ **何を**：何を構築するか。関数？コンポーネント？アプリ全体？
- ・ **なぜ**：なぜ重要か。この部品が大きな機能のどこに適合するか。
- ・ **制約**：言語、ライブラリ、性能目標、ファイルレイアウト。
- ・ **形式**：出力の形。コードブロック、ファイルリスト、diff、説明の深度。

すべてのリクエストをこれら4つのチャンクに分割します。リズムが手につくと、4つすべてを単一の文に詰め込むことができます。第2章でこれをループの一部として扱います。

以前行っていた**コードレビュー**は他の誰かのPRを読むことでした。現在行っている**要件改良**はAIが何かを作る前に自分の頭を整理することです。レビューは事後。改良は事前。この逆転が最大の変化です。

2.8 1.7 シーン—readbooks.aiからのPDFパーサーエピソード

抽象的に話すのは感じにくいので、短いシーンを紹介します。

readbooks.aiにPDFテキストエクストラクターを追加していました。最初のプロンプトはこの短さでした。

PDFテキストを抽出するRustベースのTauriコマンド。

Claudeはpdf-extractクレートを使った20行のコマンドを返しました。実行しました。英語PDFは動作しました。韓国語PDFは**壊れた文字の箱**として戻ってきました。

以前なら Stack Overflowで「rust pdf korean encoding」を検索していたでしょう。代わりに証拠を持参して再度尋ねました。

韓国語PDFが文字化けして出力されます。サンプル入力の最初のページを示します：
<壊れたテキスト30行>

3つの候補原因とそれぞれの短い修正を列挙し、

最も現実的なものを選んで、修正されたコマンドを提供してください。

Claudeは3つの候補—エンコーディング、フォント埋め込み、不足CMap—を列挙し、答えました：「この場合、ほぼ確実にCMapの問題です。純粋なpdf-extractにはここに上限があります。pdfiumを検討するか外部バイナリにシェルアウトしてください。」正しい診断でした。

ここで重要なのはClaudeが最初に失敗したことではありません。私が**証拠（壊れた出力）をプロンプトにフィードバック**し、2番目のリクエストを尖らせたことです。以前の私なら Google に「これをどう修正するか」と尋ねていたでしょう。現在の私は**正確に観察したものをAIに見せ、それで次の仮説を構築します**。

この書籍が教えることのほとんどがこのシーンにあります。短いリクエスト、証拠に基づくフィードバック、列挙された仮説、現実的な選択—そして**AIを決して神託として扱わない姿勢**。

2.9 1.8 一緒に構築する6つのプロジェクト

この書籍全体で、構築した6つのプロジェクトを素材として使います。それぞれがAIとの作業の異なる顔を示します。

2.9.1 1. Mandai —生産性ツリー + AIコーチ

Mandal-Art (9×9目標グリッド) をGTDとPomodoroと融合。全データはローカルDuckDBにスタックし、4つのプロバイダー—OpenAI、Claude、Gemini、Groq—が単一のTauriコマンドの背後にあります。このアプリは**マルチプロバイダーストリーミングパターン**の教科書です。

AIが最も間違えがちだった箇所はストリームキャンセレーションでした。ユーザーが質問を変更したとき、前のストリームは停止する必要があります。最初はUIが毎回フリーズしました。修正に3回のループが必要でした。

2.9.2 2. readbooks.ai —多言語PDF翻訳

PDFを36言語に翻訳します。Next.jsからSvelteKit 5に移植されたライブサービス。Claude、OpenAI、Gemini、DeepLが単一のtranslate_with_providerコマンドの背後に隠れています。このアプリは**フレームワーク移植**と**サードパーティAPI統合**を自分とAIの間でどう分割するかを示します。

Next.js -> SvelteKit移植を完全にAIに任せられると思っていました。できませんでした。ルーティング構造が異なり、データ取得モデルが異なり、3つの大きなループが必要でした。

2.9.3 3. トレーディングダッシュボード—19画面リアルタイムダッシュボード

WebSocket上でPython FastAPIバックエンドに接続します。AtomicUI16ベースプロキシポート管理、Cloudflare Tunnel統合、ローカルチャートレンダリングがすべてTauri内に存在します。これは**多くの画面を持つアプリをAIプロンプト間で分割する教科書**です。

19画面では、すべてを一度にAIに渡すことはできません。画面ごとに1セクションに切り、共有レイアウトとWebSocketフックを最初に置きました。その分割戦略が第13章の中心です。

2.9.4 4. RL & バックテストラボ—長時間実行Pythonプロセスコントローラー

強化学習トレーニング実行とバックテスト—数時間かかるPythonスクリプト—がTauriから起動され、stdoutがフロントエンドにストリーミングされます。これは**長寿命外部プロセス**の教科書です。

AIはここで最も弱かったです。詰まったプロセスをkill -9するコードパスを決して入れませんでした。私が個人的に問題にぶつかってから「SIGKILLを考慮する」とプロンプトで明記する必要がありました。

2.9.5 5. TUIトレーディングモニター—ターミナルダッシュボード

GUIダッシュボードと同じトレーディングデータを、ratatui でターミナルにレンダリング。ポジション、P&L、アラート—すべて単一バイナリTUI内。このプロジェクトは**TUIがどれほど速く出荷できるか**の教科書です。GUIダッシュボードに2週間かかりました。TUIモニターは1日でした。

2.9.6 6. TUIログビューアー—アイデアからツールまで2時間

RLトレーニング出力用のターミナルベースログビューアー。ライブテーリング、正規表現検索、統計抽出。AIで2時間で構築。このプロジェクトは**TUIがアイデアから動作するツールまでの最速パス**であることを証明します。

2.10 1.9 読者が持ち帰るもの

この書籍を読み終えても、すべてのTauri 2 APIを暗記することはありません。Rustエキスパートになることもありません。それは目標ではありません。

持ち帰るものは**同じ方法を使って自分のアプリを構築する能力**です。具体的には、5つのこと。

1. **アイデアをAIが解析できるリクエストに翻訳する訓練。**頭の中の「このような感じのアプリ」を実際のコードを生成するプロンプトに移動する筋力。
 2. **AI出力を判定する目。**正しいコードと間違ったコードを区別し、正しいが危険なコードを見抜くことができます。特に、Tauri 2の一般的なトラップ（パーミッション、State、IPC、非同期ミューテックス）を事前に内在化します。
 3. **停滞せずにループを続けるペース。**最初のパスが決して完璧ではないことを受け入れ、完了に向かう3〜5回の短いサイクルのリズムに落ち着きます。
 4. **6つの具体的な参照プロジェクト。**構築した実際のアプリの構造と、AIが貢献した正確な箇所と私が上書きした箇所を読みます。似たようなものを構築するときにはほぼそのまま持ち上げることができるパターン。
 5. **適切なインターフェイスを選ぶ判断。**ユーザーが重要ならGUI、スピードが重要ならTUI、監視と洗練の両方が必要なら両方。その選択がそれに続くすべてのプロンプトを形作ります。
-

2.11 1.10 2年間、数値で

手を振る議論の部分を数値で締めくくみましょう。同じ人間（私）の2020年と2026年。

指標	2020年	2026年
出荷したデスクトップアプリ	1	4 + 2 TUI
書いたコード行数（概算）	3万行	10万行
手でタイプした割合	95%	10%未満
1日あたりプロンプト数	0	40 -80
1日あたりコミット数	2 -3	8 -15
主要言語	TypeScript	TypeScript + Rust + Python

数値はGitHubとシェル履歴から抽出した大まかな推定です。厳密なベンチマークではなく――**規模感**です。2つのことが目立ちます。

第一に、**新言語（Rust、Python）の割合が増加したにも関わらず生産性が上昇**しました。以前ならRustを学ぶのに半年食われていたでしょう。AIがその半年を「必要なときにいつでも尋ねる数分、検索クエリのように」に圧縮しました。

第二に、**コミット数の上昇**は私が小さなループを保っている証拠です。1日10コミットは1コミット1時間未満を意味します。それがAIが有用であり続ける唯一のペースです。

2.12 1.11 始める前のセルフチェック

この書籍から最大限得るには、あなたからの何かが必要です。これら5つを実行してください。

- ☐ git、npm/pnpm、cargoがターミナルでつまづかない。
- ☐ 少なくとも1つのAIコーディングツール（Claude Code、Cursor、Copilotなど）をインストールしている。

- TypeScriptを問題なく読める。Rust知識はオプション。
- 過去3ヶ月で「馴染みのないライブラリを1時間以内で接続」を少なくとも1回行った。
- 毎日少なくとも30分の集中時間を確保できる。

5つすべてがチェックされていれば、第2章に直行してください。3つ以下なら、第4章の環境設定を最初に読み、マシンを準備してください。

2.13 1.12 この本がカバーしないこと

境界について正直であることです。

- ・ **一般的なプロンプトエンジニアリング**は対象外です。この本は、デスクトップアプリ構築の文脈でのプロンプトのみをカバーします。文章作成、画像生成、RAGについては別の本を読んでください。
- ・ **AIモデルの内部構造**は対象外です。「なぜClaudeがOpenAIよりRustを上手に書くのか」のような質問は範囲外です。
- ・ **公式Tauri 2ドキュメントのコピー**ではありません。最小限の概念のみを説明し、「この概念をAIとどう使ったか」に時間を費やします。
- ・ **初回で成功する話**はありません。すべての章に私が間違っていたシーン、AIが間違っていたシーン、ループを再実行したシーンが含まれます。それが核となる材料です。

2.14 要約

- ・ 2026年に私は本業を持ち子どもを育てながら4つのTauri 2アプリをリリースしました。時間があつたからではなく、**時間がなく、AIエージェントがその隙間を埋めてくれた**からです。
- ・ 核心的変化は技術スタックではありません。**タイピングをAIエージェントに委任した**ことです。

- ・ 開発は「私がコードを書く」を意味していました。今は「私が決定を下す」を意味します。
 - ・ 「AI生成コードを読めるだけでよい」は半分正しいです。**読解ハードル**は以前より遙かに高くなりました。
 - ・ 役割はコードレビュー（事後）から要件精緻化（事前）にシフトしました。プロンプトを書くことが私の一日で最も長い活動です。
 - ・ この書籍は6つのプロジェクト—Mandai、readbooks.ai、トレーディングダッシュボード、RL & バックテストラボ、TUIトレーディングモニター、TUIログビューアー—を使用して、同じ方法を自分のアプリに適用できるよう支援します。
-

2.15 1.12 この書籍がカバーしないもの

境界について正直に。

- ・ **一般プロンプトエンジニアリング**は範囲外です。この書籍はデスクトップアプリ構築の文脈でのみプロンプトをカバーします。執筆、画像生成、RAGについては他の書籍を読んでください。
 - ・ **AIモデルの内部構造**は範囲外です。なぜClaudeがOpenAIよりRustを上手く書くかといった質問は範囲外です。
 - ・ **公式Tauri 2ドキュメントのコピー**ではありません。最小限の概念のみを説明し、「AIでこの概念をどう使ったか」に時間を費やします。
 - ・ **最初で成功する話はありません**。すべての章で、私が間違っていたシーン、AIが間違っていたシーン、ループを再実行したシーンが含まれています。それが核心材料です。
-

2.16 まとめ

- ・ 2026年に私は本業と子育てをしながら4つのTauri 2アプリを出荷しました。時間があつたからではなく、**時間がなく、AIエージェントがギャップを埋めてくれた**からです。
- ・ 核心的な変化は技術スタックではありません。**タイピングをAIエージェントに委任した**ことです。
- ・ 開発は「私がコードを書く人」から「私が決定を下す人」に変わりました。
- ・ 「AI コードを読むだけで十分」は半分正しいです。**読み取り基準**は以前よりもはるかに高くなっています。
- ・ 役割はコードレビュー（事後）から要件改良（事前）にシフトしました。プロンプトを書くことが私の1日で最も長い活動です。
- ・ この書籍は6つのプロジェクト—Mandai、readbooks.ai、トレーディングダッシュボード、RL & バックテストラボ、TUIトレーディングモニター、TUIログビューアー—を使用して、同じ方法を自分のアプリに適用できるよう支援します。